book-qr-manager 学習ノート

学習日：2026年8月30日
テーマ：DTO・Repositoryの`create()`/`save()`・async/await・Controller・POST API

---

1. 今日の学習内容

今日は昨日作成した`LocationsModule`と`LocationsService`を使って、**Locationを実際にPostgreSQLへ登録するPOST API**を作成しました。

主に学習した内容は次のとおりです。

- DTOとは何か
- `CreateLocationDto`の作成
- TypeScriptの`?`（任意プロパティ）
- `Repository.create()`の役割
- `Repository.save()`の役割
- `async` / `await` / `Promise`の基本
- Controllerとは何か
- `@Controller()` / `@Post()` / `@Body()`の役割
- ControllerをModuleへ登録する方法
- PostmanからPOST APIを実行する方法
- `Content-Type: application/json`の重要性
- PostgreSQLへのINSERT確認

---

2. DTOとは？

DTOは **Data Transfer Object** の略です。

簡単にいうと、

> **APIで受け取るデータの形を定義するためのクラス**

です。

Location登録では、次のファイルを作成しました。

```
src/
└── locations/
    └── dto/
        └── create-location.dto.ts
```

作成したDTOは次のとおりです。

```
export class CreateLocationDto {
  name: string;

  shelf?: string;

  row?: string;

  column?: string;

  notes?: string;
}
```

例えばPostmanから、

```
{
  "name": "自宅本棚",
  "shelf": "本棚A",
  "row": "2段目",
  "column": "右側",
  "notes": "技術書"
}
```

というデータを送信すると、この入力データを`CreateLocationDto`として受け取ります。

---

3. DTOとEntityの違い

DTOとEntityは似ていますが、役割が違います。

```
外部から送られてきたデータ
        ↓
CreateLocationDto
「APIではどんな入力を受け取る？」
        ↓
Service
        ↓
Location Entity
「DBではどんなデータとして扱う？」
        ↓
Repository
        ↓
PostgreSQL
```

まずは次のように覚えます。

- **DTO**：APIとのデータの受け渡し

- **Entity**：データベースとのデータの受け渡し

---

4. ? の意味

DTOでは、

```
name: string;
```

と、

```
shelf?: string;
```

の2種類を書きました。

`name: string`は、`name`が必須であることを表します。

一方、

```
shelf?: string;
```

の`?`は、

> **このプロパティは存在しなくてもよい**

というTypeScriptの意味です。

そのため、Location登録時に、

```
{
  "name": "自宅本棚"
}
```

だけを送ることも、型の上では可能です。

---

5. `Repository.create()`とは？

`LocationsService`にはLocation Repositoryを注入してあります。

```
constructor(
  @InjectRepository(Location)
```

```
    private readonly locationRepository: Repository<Location>,
  ) {}
```

そのRepositoryを使って、

```
  const location =
    this.locationRepository.create(createLocationDto);
```

としました。

`create()`の役割は、

> **受け取ったデータからLocation Entityのオブジェクトを作る**

ことです。

重要なのは、`create()`を実行しただけでは**PostgreSQLにはまだ保存されていない**ということです。

```
CreateLocationDto
        ↓
repository.create()
        ↓
Locationオブジェクト
        ↓
まだDBには保存されていない
```

---

6. `Repository.save()`とは？

作成したLocationを実際にDBへ保存するために、

```
  const savedLocation =
    await this.locationRepository.save(location);
```

を使用しました。

`save()`を実行すると、今回のような新規データの場合はPostgreSQLへINSERTされます。

```
repository.create()
        ↓
Location Entityを作る
        ↓
repository.save()
```

```
        ↓
PostgreSQLへ保存
```

今日の重要ポイントの一つです。

> `create()` = **Entityを作る**
>
> `save()` = **DBへ保存する**

---

7. 完成した`LocationsService.create()`

今日作成した登録処理は次の形です。

```
@Injectable()
export class LocationsService {
  constructor(
    @InjectRepository(Location)
    private readonly locationRepository: Repository<Location>,
  ) {}

  async create(
    createLocationDto: CreateLocationDto,
  ): Promise<Location> {
    const location =
      this.locationRepository.create(createLocationDto);

    const savedLocation =
      await this.locationRepository.save(location);

    return savedLocation;
  }
}
```

処理の流れは、

```
CreateLocationDto
        ↓
repository.create()
        ↓
Location Entity
        ↓
repository.save()
        ↓
PostgreSQL
        ↓
保存されたLocationをreturn
```

です。

---

8. asyncとは？

今回、

```
async create(...): Promise<Location>
```

と書きました。

asyncは、

> **この関数では非同期処理を扱う**

ということを表します。

データベースへのアクセスは、結果が返ってくるまで時間がかかる処理です。

そのため、DB処理ではasync / awaitをよく使用します。

---

9. awaitとは？

今回、

```
const savedLocation =
  await this.locationRepository.save(location);
```

としました。

awaitは、

> **この非同期処理が完了して結果が返ってくるのを待つ**

という意味です。

イメージは次のとおりです。

```
save(location)
      ↓
PostgreSQLへ保存を依頼
      ↓
awaitで待つ
      ↓
保存完了
      ↓
```

```
savedLocationに結果が入る
      ↓
return
```

---

10. `Promise<Location>`とは？

`async`を付けた関数はPromiseを返します。

```
async create(...): Promise<Location>
```

`Promise<Location>`は、初心者の段階では、

> **処理が完了したらLocationが得られる**

と考えれば大丈夫です。

整理すると、

- `async`：非同期処理を扱う関数
- `await`：非同期処理の完了を待つ
- `Promise<Location>`：処理完了後にLocationが得られる

という関係です。

---

11. Controllerとは？

次に`LocationsController`を作成しました。

Controllerは、

> **HTTPリクエストを受け付ける窓口**

です。

今回作りたかった処理は、

```
Postman
   ↓
POST /locations
   ↓
LocationsController
   ↓
LocationsService
   ↓
Location Repository
```

```
    ↓
PostgreSQL
```

という流れです。

---

12. 作成したLocationsController

```ts
import { Body, Controller, Post } from '@nestjs/common';
import { LocationsService } from './locations.service';
import { CreateLocationDto } from './dto/create-location.dto';
import { Location } from './location.entity';

@Controller('locations')
export class LocationsController {
  constructor(
    private readonly locationsService: LocationsService,
  ) {}

  @Post()
  async create(
    @Body() createLocationDto: CreateLocationDto,
  ): Promise<Location> {
    return this.locationsService.create(createLocationDto);
  }
}
```

---

13. @Controller('locations')

```ts
@Controller('locations')
```

によって、このControllerの基本URLを、

```
/locations
```

にしています。

---

14. @Post()

```
@Post()
```

は、

> **POSTリクエストを受け付ける**

という意味です。

`@Controller('locations')`と組み合わせることで、

```
POST /locations
```

というAPIになります。

---

15. @Body()とは？

```
@Body() createLocationDto: CreateLocationDto
```

の`@Body()`は、

> **HTTPリクエストのBody（本文）を取得する**

ためのデコレーターです。

Postmanから、

```
{
  "name": "自宅本棚",
  "shelf": "本棚A"
}
```

を送ると、そのBodyを`createLocationDto`として受け取ります。

---

16. ControllerをModuleへ登録

Controllerを作成しただけではNestJSから利用されません。

`LocationsModule`へ登録しました。

```
@Module({
  imports: [TypeOrmModule.forFeature([Location])],
  providers: [LocationsService],
  controllers: [LocationsController],
})
export class LocationsModule {}
```

それぞれの役割は、

```
LocationsModule
│
├── imports
│   └── Location Repositoryを利用可能にする
│
├── providers
│   └── LocationsServiceを登録
│
└── controllers
    └── LocationsControllerを登録
```

です。

---

17. PostmanでPOST APIを実行

バックエンドを、

```
npm run start:dev
```

で起動しました。

NestJSのログには、

```
Mapped {/locations, POST} route
Nest application successfully started
```

と表示され、`POST /locations`が正常に登録されていることを確認できました。

Postmanから、

```
POST http://localhost:3000/locations
```

へJSONを送信しました。

---

18. 今日発生した500エラー

最初の実行では、

```
QueryFailedError:
locationの列nameのNULL値が非NULL制約に違反
```

というエラーが発生しました。

SQLにも、

```
VALUES (DEFAULT, DEFAULT, DEFAULT, ...)
```

と表示されていました。

これは、Postmanから送ったBodyがNestJSでJSONとして正しく受け取れておらず、`name`などが`undefined`になっていたためです。

---

19. 原因：PostmanのBodyがTextだった

Postmanでは、

```
Body
→ raw
→ Text
```

となっていました。

これを、

```
Body
→ raw
→ JSON
```

へ変更しました。

JSONとして送る場合は、

```
Content-Type: application/json
```

が重要です。

Postmanで`raw → JSON`を選択すると、通常は自動的に設定されます。

---

20. 登録成功

JSONへ変更して再度POSTすると、

```
Status: 201 Created
```

となりました。

`201 Created`は、

> **新しいデータの作成に成功した**

ことを表すHTTPステータスです。

さらにpgAdminで`public.location`テーブルを確認し、

```
name      自宅本棚
shelf     本棚A
row       2段目
column    右側
```

などのデータが実際に登録されていることを確認しました。

複数行登録されていたのは、PostmanでPOSTを複数回実行したためです。

---

21. 今日完成した処理の全体像

今日一番重要な流れです。

```
Postman
   │
   │ POST /locations
   │ JSON
   ↓
LocationsController
   │
```

```
    │ @Body()
    ↓
CreateLocationDto
    │
    ↓
LocationsService.create()
    │
    ├── repository.create()
    │       ↓
    │   Location Entityを作る
    │
    └── repository.save()
            ↓
       PostgreSQL
            ↓
      locationテーブル
```

昨日学習した、

```
Module
→ Service
→ Repository
→ Entity
→ PostgreSQL
```

に、今日はControllerとDTOが加わりました。

より実際のAPIに近い形では、

```
Controller
    ↓
DTO
    ↓
Service
    ↓
Repository
    ↓
Entity
    ↓
PostgreSQL
```

という流れをイメージしておきます。

---

22. 今日の重要ポイント

今日特に覚えておきたいのは次の内容です。

DTO

APIで受け取るデータの形を定義する。

`repository.create()`

受け取ったデータからEntityオブジェクトを作る。
まだDBには保存しない。

`repository.save()`

Entityを実際にDBへ保存する。

`async`

非同期処理を扱う関数であることを表す。

`await`

非同期処理の完了を待って結果を受け取る。

Controller

HTTPリクエストを受け付け、Serviceへ処理を渡す。

`@Body()`

HTTPリクエストのBodyを取得する。

`201 Created`

新しいデータの作成に成功したことを表す。

---

23. 次回に向けて

現在、Locationについては、

```
POST /locations
```

で新しい保管場所を登録できるようになりました。

次回は、この登録APIを土台にして次の処理へ進めていきます。

今回学習したController・Service・Repositoryの流れは、今後Book APIを作るときにも同じ考え方を使います。

復習用ひとこと

Controllerがリクエストを受け取り、DTOで入力データを扱い、ServiceがRepositoryを使ってDBへ保存する。

そして、

`create()`はEntityを作るだけ、`save()`で初めてDBに保存される。

この2つを今日の中心として覚えておきます。